ML TEAM — SPRING 2026

Practice Worksheet: Neural Networks · CNNs · Transformers · Attention · RL · Q-Learning

Name: Karan Singh

Team:

Section 1 — Neural Networks (/ 20 pts)

Q1 **[Calculation]** A neuron has weights w = [3, -2, 1], bias b = -1, and input x = [2, 1, 4]. Compute the raw output z = w·x + b, then apply ReLU. Show your work.

$z = \begin{bmatrix} 3 \\ -2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix} + (-1)$

$= 6 - 2 + 4 - 1$

$= 7$

$ReLU = \max(0, 7) =$

Q2 **[Short Answer]** A network has 3 layers with dimensions: input (784) → hidden (128) → hidden (64) → output (10). How many learnable parameters are in the weight matrices only (ignore biases)?

$W_1 = 784 \times 128 = 100{,}352$

$W_2 = 128 \times 64 = 8{,}192$

$W_3 = 64 \times 10 = 642$

$W_1 + W_2 + W_3 = 109{,}184$

Q3 **[Explain]** Without activation functions, a 10-layer neural network collapses to a single linear transformation. Explain why using matrix algebra.

Suppose you has 10 layers of weights. Then, when we pass our input in we will get $(W_{10} \cdot W_9 \cdots W_1)x$. This is the same as a one layer model.

Q4 **[Short Answer]** You're training a classifier and notice the training loss is very low but validation loss is very high. Name this problem, and give two ways to fix it using what you know about neural networks.

This is overfitting. We can fix this with randomly zeroing out neurons during training and using fewer parameters.

Section 2 — Convolutional Neural Networks (/ 20 pts)

Q5 **[Calculation]** Apply the 1D filter W = [1, 0, -1] with stride 1 to input X = [3, 1, 4, 1, 5, 9]. Write the full output and explain what pattern this filter detects.

$1 \cdot 3 + 0 \cdot 1 + -1 \cdot 4 = -1$

$1 \cdot 1 + 0 \cdot 4 + -1 \cdot 1 = 0$

$1 \cdot 4 + 0 \cdot 1 + -1 \cdot 5 = -1$

$1 \cdot 1 + 0 \cdot 5 + -1 \cdot 9 = -8$

$[-1, 0, -1, -8]$

This filter detects edge

Q6 **[Calculation]** An image is 64×64. You apply: (1) Conv with 3×3 filter, stride 1, no padding. (2) 2×2 max pool, stride 2. (3) Conv with 3×3 filter, stride 1, no padding. What is the spatial size after each step?

$O = \frac{(V - R + 2Z)}{S} + 1$

(1) $\frac{(64-3)}{1} + 1 = 62$

(2) $\frac{62}{2} = 31$

(3) $\left(\frac{31-3}{1}\right) + 1 = 29$

Q7 **[Explain]** Why do early CNN layers detect edges while deep layers detect semantic concepts like 'face' or 'wheel'? Explain in terms of receptive fields.

A neuron in the first layer only sees a patch of a image. The next layer will see a patch from last layers output. Each layer sees patterns from the layer before and more.

Q8 **[Compare]** A fully connected layer connecting 64×64 input to 1000 neurons has how many weights? A convolutional layer with 32 filters of size 3×3 on the same input has how many weights? Why is this difference important?

$64 \times 64 \times 1000 = 4{,}096{,}000$

$32 \times 3 \times 3 = 288$

Same info in fewer dimensions.

Section 3 — Transformers & Attention (/ 20 pts)

Q9 **[Explain]** Consider: 'I saw the man with the telescope.' Explain how attention assigns different weights to 'man' and 'telescope' when computing the contextual embedding for 'with'. What does this tell us about the sentence's ambiguity?

This sentence is ambiguous. Depending on which weight 'man' and 'telescope' got assigned, the meaning of the sentence changes.

Q10 **[Calculation]** You have Q = [1, 0], K = [[1, 0], [0, 1], [-1, 0]], d_k = 2. Compute the raw attention scores, scale by 1/√d_k, and apply softmax. Which key does Q attend to most?

$Q \cdot K_1 = 1 \cdot 1 + 0 \cdot 0 = 1$

$Q \cdot K_2 = 0 \cdot 1 + 1 \cdot 0 = 0$

$Q \cdot K_3 = 1 \cdot -1 + 0 \cdot 0 = -1$

$\frac{1}{\sqrt{2}} = 0.707$

$[0.707, 0, -0.707]$

$e^{0.707} = 2.028$

$e^{0} = 1$

$e^{-0.707} = 0.493$

Sum = 3.521

weights = [0.576, 0.284, 0.140]

Q attends to K_1 the most

Q11 **[Short Answer]** A transformer is trained on the sentence 'I love to eat french fries with ketchup.' During training it predicts every word autoregressively. Draw the causal mask as a 9×9 grid (use 1 = attend, 0 = masked) for the first 4 tokens.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Q12 **[Compare]** BERT uses bidirectional attention. GPT uses causal (left-to-right) attention. For each model, explain which task it is better suited for and why the masking choice directly causes this.

In BERT, every token tends to every other token. This is best for understanding tasks. In GPT we can only see past tok

-ens. This is best for generation tasks.

Section 4 — Reinforcement Learning (/ 20 pts)

Q13 **[Calculation]** $\gamma = 0.8$. An agent receives rewards $r_0=0$, $r_1=0$, $r_2=5$, $r_3=0$, $r_4=10$ then the episode ends. Compute the discounted return G_0 starting from t=0.

$G_0 = 0 + 0.8 \cdot 0 + 0.8^2 \cdot 5 + 0.8^3 \cdot 0 + 0.8^4 \cdot 10$

$= 7.296$

Q14 **[Short Answer]** Define the Markov property. Give one example of a real-world problem where the Markov property holds, and one where it does NOT hold.

The future state depends only on the current state. Holds for chess, does not hold when current state does not have all information.

Q15 **[Explain]** What is the credit assignment problem? Use a chess analogy. How does the Bellman equation help solve it?

Rewards are delayed. So which actions deserve credit?
Bellman has the answer, reward propagates backward through the value function.

Q16 **[Compare]** An agent uses ε-greedy with ε = 0.1. (a) What fraction of actions are random? (b) If the agent is near a cliff with -100 reward, what is the expected penalty per step from random actions? (c) Should ε be higher early or late in training? Why?

(a) 10%

(b) If 4 actions, $0.1 \cdot 4 \cdot -100 = -2.5$

(c) Higher early since you know nothing at the start

Section 5 — Q-Learning (/ 20 pts)

Q17 **[Calculation]** Current Q(s,a) = 4. You observe r = 8, next state s' has Q(s', left) = 6 and Q(s', right) = 10. α = 0.3, γ = 0.9. Compute the new Q(s,a). Show the TD target and TD error.

TD target = $8 + 0.9 \cdot 10 = 17$

TD error = $17 - 4 = 13$

New $Q(s,a) = 4 + 0.3 \cdot 13 = 7.9$

Q18 **[Trace]** 4-state chain: S0→S1→S2→S3(terminal). r(S2→S3)=20, all others 0. α=0.5, γ=1.0, Q initialized to 0. Trace Episodes 1 and 2 fully. Fill in the table below.

Step	Transition	Reward	Q Update Formula	Q(S1)	Q(S2)
Ep1 Step1 (S0→S1)	$S_0 \to S_1$	0	0	0	0
Ep1 Step2 (S1→S2)	$S_1 \to S_2$	0	0	0	0
Ep1 Step3 (S2→S3)	$S_2 \to S_3$	20	10	0	10
Ep2 Step1 (S0→S1)	$S_0 \to S_1$	0	0	0	10
Ep2 Step2 (S1→S2)	$S_1 \to S_2$	0	5	5	10
Ep2 Step3 (S2→S3)	$S_2 \to S_3$	20	15	5	15

Q19 **[Short Answer]** Q-learning is called 'off-policy.' What does this mean? Write out the SARSA update rule and explain the one key difference from Q-learning's update rule.

The policy used to collect data differs from the policy being optimized.

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + Q(s',a') - Q(s,a)$

Sarsa bootstraps from what agent did next, not next best action

Q20 **[Explain]** A Q-table has 10,000 states and 5 actions. Describe one situation where this is fine and one where this is completely unworkable. What is the standard fix for the unworkable case?

Works → Standard discrete states

Breaks → High dimension inputs

Fix, use DQN

ANSWER KEY

For instructor use — do not distribute to students before worksheet completion

Section 1 — Neural Networks

Q1: Compute z = w·x + b then ReLU.

z = (3)(2) + (-2)(1) + (1)(4) + (-1) = 6 - 2 + 4 - 1 = 7

ReLU(7) = max(0, 7) = 7

Q2: Count weights across all weight matrices.

Layer 1: 784 × 128 = 100,352

Layer 2: 128 × 64 = 8,192

Layer 3: 64 × 10 = 640

Total = 109,184 weights (biases not counted)

Q3: Explain why depth without activation collapses to linearity.

Without activations, each layer is $f(x) = W_i x$. Composing: $W_3(W_2(W_1 x)) = (W_3 W_2 W_1)x$ — just one matrix multiply.

You could replace any deep linear network with a single matrix $M = W_3 W_2 W_1$. No depth benefit at all.

Activation functions (e.g. ReLU) break this collapse by introducing nonlinearity between layers.

Q4: Name the problem and give two fixes.

Problem: Overfitting (high variance). The model has memorized training data but doesn't generalize.

Fix 1: Dropout — randomly zero out neurons during training, forcing redundant feature learning.

Fix 2: Reduce model size (fewer layers/neurons) — less capacity = less memorization.

Also valid: L2 regularization (weight decay), early stopping, more training data.

Section 2 — CNNs

Q5: Convolve W = [1, 0, -1] over X = [3, 1, 4, 1, 5, 9].

Position 0: (1)(3) + (0)(1) + (-1)(4) = 3 - 4 = -1

Position 1: (1)(1) + (0)(4) + (-1)(1) = 1 - 1 = 0

Position 2: (1)(4) + (0)(1) + (-1)(5) = 4 - 5 = -1

Position 3: (1)(1) + (0)(5) + (-1)(9) = 1 - 9 = -8

Output: [-1, 0, -1, -8]

This filter computes $x_i - x_{i_{+2}}$. Large negative values indicate a big rise (e.g. -8 at position 3: 1 vs 9). It detects right-edges / rising slopes.

Q6: Track spatial size through three operations.

Start: 64×64

Step 1 — Conv 3×3, stride 1, no pad: (64-3)/1 + 1 = 62 → 62×62

Step 2 — MaxPool 2×2, stride 2: 62/2 = 31 → 31×31

Step 3 — Conv 3×3, stride 1, no pad: (31-3)/1 + 1 = 29 → 29×29

Q7: Explain hierarchical feature detection via receptive fields.

A neuron in layer 1 sees only a 3×3 patch of the original image — just enough to detect an edge.

A neuron in layer 2 sees a 3×3 patch of layer 1's output, which itself covers a 5×5 region of the original image.

Each layer's effective receptive field grows, so deeper neurons integrate information from larger image regions.

This allows them to detect combinations of edges (textures), then combinations of textures (shapes), then shapes (objects).

Q8: Compare parameter counts: FC vs. Conv.

Fully connected (64×64 → 1000): 4096 × 1000 = 4,096,000 weights

Conv (32 filters, 3×3, 1 input channel): 32 × 3 × 3 × 1 = 288 weights

Ratio: 4,096,000 / 288 ≈ 14,222× fewer parameters in the conv layer.

This matters because: fewer params = less memory, faster training, less overfitting. The same 3×3 filter is reused (shared) across the entire image — a massive efficiency gain.

Section 3 — Transformers & Attention

Q9: Attention and the 'with the telescope' ambiguity.

The sentence is ambiguous: did I see the man using a telescope, or see the man who had a telescope?

When computing the embedding for 'with', attention scores over all other words. If 'saw' has a higher key-query match with 'with', attention leans toward 'I saw [man] with [telescope]' (instrument reading). If 'man' matches more strongly, it leans toward the modifier reading. Different attention heads can hold both interpretations simultaneously — this is why multiple heads are useful. An RNN would compress prior context into a single hidden state, losing this fine-grained ambiguity.

Q10: Compute scaled dot-product attention scores and softmax.

Q = [1, 0], K = [[1,0],[0,1],[-1,0]]

Raw scores: $Q \cdot K_1^T = 1$, $Q \cdot K_2^T = 0$, $Q \cdot K_3^T = -1$

Scaled (÷√2): [1/√2, 0, -1/√2] ≈ [0.707, 0, -0.707]

Softmax: e^0.707 ≈ 2.028, e^0 = 1, e^-0.707 ≈ 0.493; sum ≈ 3.521

Weights ≈ [0.576, 0.284, 0.140]

Q attends most strongly to K_1 = [1,0] (same direction as Q). This makes sense: dot product is maximized when vectors are aligned.

Q11: Causal mask for first 4 tokens (I / love / to / eat).

4×4 causal mask (rows = query token, cols = key token):

```
      I   love  to   eat

I    [ 1    0    0    0 ]

love [ 1    1    0    0 ]

to   [ 1    1    1    0 ]

eat  [ 1    1    1    1 ]
```

1 = can attend, 0 = masked (-∞ before softmax, 0 after). Each token attends to itself and all prior tokens only.

Q12: BERT bidirectional vs. GPT causal attention.

BERT (bidirectional): every token attends to every other token. Best for tasks requiring full context understanding: classification, NER, question answering. Cannot generate text autoregressively — it sees the whole sequence at once.

GPT (causal): each token only attends to past tokens. Best for generation tasks: text completion, chatbots, summarization. The masking choice directly enables generation: at inference time, you only have past tokens, so masking future ones during training matches the actual generation setting.

Section 4 — Reinforcement Learning

Q13: Compute G_0 with γ=0.8, rewards [0,0,5,0,10].

$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \gamma^4 r_4$

$= 0 + 0.8(0) + 0.64(5) + 0.512(0) + 0.4096(10)$

$= 0 + 0 + 3.2 + 0 + 4.096$

$= 7.296$

Q14: Markov property: definition and examples.

Definition: The future state depends only on the current state, not on the history of past states. $P(s_{t+1}|s_t,a_t) = P(s_{t+1}|s_0,...,s_t,a_0,...,a_t)$.

Holds: Chess — the full board position captures all relevant game information. The history of how we got there doesn't matter.

Does NOT hold: Poker with hidden cards — the current visible state doesn't contain the full information (opponents' hole cards, betting history affects inferences). Also: medical diagnosis without full patient history.

Q15: Credit assignment problem and Bellman equation.

Problem: In a chess game, you win on move 40 with +1 reward. Which of your 40 moves actually caused the win? All prior moves got reward 0. The algorithm must figure out which early decisions led to the eventual success.

Bellman helps by recursively propagating value backward: $V(s) = E[r + \gamma V(s')]$. Each state's value is defined in terms of its successor's value. This creates a chain: the terminal reward (+1) propagates back one step per update until all states reflect the reward they lead to.

Q16: ε-greedy with ε=0.1 near a cliff.

(a) 10% of actions are random (ε = 0.1).

(b) If there are 4 possible actions and one leads off the cliff: P(cliff from random) = 1/4. Expected penalty per step from ε-greedy = 0.1 × (1/4) × (-100) = -2.5 per step on average.

(c) ε should be higher early (more exploration — you haven't learned anything yet, so random actions gather useful new information) and lower late (more exploitation — you've learned a good policy, random actions just hurt performance). Common strategy: ε-decay from 1.0 to 0.01 over training.

Section 5 — Q-Learning

Q17: Q-learning update: Q(s,a)=4, r=8, max Q(s')=10, α=0.3, γ=0.9.

TD target = r + γ · max_a' Q(s',a') = 8 + 0.9 × 10 = 8 + 9 = 17

TD error = target - Q(s,a) = 17 - 4 = 13

New Q(s,a) = 4 + 0.3 × 13 = 4 + 3.9 = 7.9

Q18: 4-state chain trace: S0→S1→S2→S3, r(S2→S3)=20, α=0.5, γ=1.0.

Ep1 Step1 (S0→S1): Q(S0) ← 0+0.5[0+Q(S1)-0]=0. Q(S1)=—

Ep1 Step2 (S1→S2): Q(S1) ← 0+0.5[0+Q(S2)-0]=0. Q(S2)=—

Ep1 Step3 (S2→S3): Q(S2) ← 0+0.5[20+0-0]=10. Q(S1)=0, Q(S2)=10

Ep2 Step1 (S0→S1): Q(S0) ← 0+0.5[0+Q(S1)-0]=0. Q(S1)=0

Ep2 Step2 (S1→S2): Q(S1) ← 0+0.5[0+10-0]=5. Q(S1)=5

Ep2 Step3 (S2→S3): Q(S2) ← 10+0.5[20+0-10]=15. Q(S2)=15

Key: credit propagates back one state per episode. After many episodes Q(S0)→20, Q(S1)→20, Q(S2)→20.

Q19: Off-policy Q-learning vs. SARSA.

Off-policy: the policy used to collect experience (behavior policy, e.g. ε-greedy) differs from the policy being optimized (target policy, greedy). Q-learning always updates toward the greedy max, regardless of what action was taken.

Q-learning update: Q(s,a) ← Q(s,a) + α[r + γ max_a' Q(s',a') - Q(s,a)]

SARSA update: Q(s,a) ← Q(s,a) + α[r + γ Q(s',a') - Q(s,a)]

Key difference: Q-learning uses max Q(s') (the best possible action). SARSA uses Q(s',a') where a' is the ACTUAL next action taken. SARSA is on-policy — it accounts for exploration in its update.

Q20: Q-table scalability: when it works vs. when it breaks.

Works fine: simple environments with discrete state and action spaces. A grid world with 10,000 positions and 5 movement directions is easily represented. The table has 50,000 entries — trivially small.

Completely unworkable: video game pixels. Atari has a 210×160×3 pixel screen — approximately 10^68 possible states. A Q-table would require more entries than atoms in the universe. You cannot store, let alone update, such a table.

Standard fix: Deep Q-Network (DQN). Replace the Q-table with a neural network Q(s,a;θ) that takes raw pixels as input and outputs Q-values for all actions. The network generalizes across similar states instead of memorizing each one.